

3D Gaussian Splatting Training Pipeline

This notebook implements a complete training pipeline for 3D Gaussian Splatting. It combines:

1. **Differentiable Rasterizer:** Projects 3D Gaussians to 2D images.
2. **Adaptive Density Control:** Clones/Splits/Prunes Gaussians during training.
3. **Multi-View Optimization:** Trains on multiple views of a 3D scene to recover 3D structure.
4. **Robust Loss:** Uses a combination of L1 and D-SSIM loss.

The Scene

To make this standalone, we generate a **Synthetic Dataset** on the fly:

- **Ground Truth:** A scene composed of 3 distinct colored spheres (Red, Green, Blue).
- **Cameras:** We generate circular camera paths looking at the center.
- **Task:** The model initializes with random sparse points and must learn to reconstruct the spheres from the rendered views.

```
In [ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
import math
import numpy as np
import matplotlib.pyplot as plt
import random

# Set seed
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

device = torch.device('cpu') # Use CPU for safety and simplicity
print(f"Using device: {device}")
```

```
In [ ]: # --- MATH & RASTERIZER PRIMITIVES ---

def build_rotation(r):
    norm = torch.sqrt(r[:,0]*r[:,0] + r[:,1]*r[:,1] + r[:,2]*r[:,2] + r[:,3]*r[:,3])
    q = r / norm[:, None]
    R = torch.zeros((q.shape[0], 3, 3), device=device)
    r = q[:, 0]; x = q[:, 1]; y = q[:, 2]; z = q[:, 3]
    R[:, 0, 0] = 1 - 2 * (y*y + z*z); R[:, 0, 1] = 2 * (x*y - r*z); R[:, 0, 2] = 2 * (x*z + r*y);
    R[:, 1, 0] = 2 * (x*y + r*z); R[:, 1, 1] = 1 - 2 * (x*x + z*z); R[:, 1, 2] = 2 * (y*z - r*x);
    R[:, 2, 0] = 2 * (x*z - r*y); R[:, 2, 1] = 2 * (y*z + r*x); R[:, 2, 2] = 1 - 2 * (x*x + y*y);
    return R
```

```

def build_scaling(s):
    S = torch.zeros((s.shape[0], 3, 3), device=device)
    S[:, 0, 0] = s[:, 0]; S[:, 1, 1] = s[:, 1]; S[:, 2, 2] = s[:, 2]
    return S

def compute_covariance_3d(scaling, quaternion):
    R = build_rotation(quaternion)
    S = build_scaling(scaling)
    M = torch.bmm(R, S)
    return torch.bmm(M, M.transpose(1, 2))

def compute_2d_covariance(covariance_3d, view_matrix, world_pos, width, height):
    R_view = view_matrix[:3, :3]
    t_view = view_matrix[:3, 3]
    pos_view = torch.matmul(world_pos, R_view.T) + t_view
    x, y, z = pos_view[:, 0], pos_view[:, 1], pos_view[:, 2]

    focal_x = width / (2.0 * tan_fovx)
    focal_y = height / (2.0 * tan_fovy)
    z = torch.clamp(z, min=0.001)

    J = torch.zeros((world_pos.shape[0], 2, 3), device=device)
    J[:, 0, 0] = focal_x / z; J[:, 0, 2] = -(focal_x * x) / (z * z)
    J[:, 1, 1] = focal_y / z; J[:, 1, 2] = -(focal_y * y) / (z * z)

    W = R_view.unsqueeze(0).expand(world_pos.shape[0], 3, 3)
    T = torch.bmm(J, W)
    cov2d = torch.bmm(T, torch.bmm(covariance_3d, T.transpose(1, 2)))
    cov2d[:, 0, 0] += 0.3; cov2d[:, 1, 1] += 0.3
    return cov2d, pos_view

def differentiable_rasterizer(world_pos, colors, opacities, scales, rots, width, height, fov_y):
    tan_fovy = math.tan(math.radians(fov_y) * 0.5)
    tan_fovx = tan_fovy * (width / height)

    # Invert View Matrix (World-to-Camera -> Camera-to-World) typically?
    # Actually our math expects World-to-Camera (R, t).
    # If passed matrix is Camera-to-World (Pose), we invert.
    # We will assume input IS World-to-Camera for simplicity.

    cov3d = compute_covariance_3d(scales, rots)
    cov2d, pos_view = compute_2d_covariance(cov3d, view_matrix, world_pos, width, height)

    focal_x = width / (2.0 * tan_fovx)
    focal_y = height / (2.0 * tan_fovy)
    screen_x = (pos_view[:, 0] / pos_view[:, 2]) * focal_x + width / 2.0
    screen_y = (pos_view[:, 1] / pos_view[:, 2]) * focal_y + height / 2.0

    depths = pos_view[:, 2]
    sorted_idx = torch.argsort(depths, descending=True)

    y_grid, x_grid = torch.meshgrid(torch.arange(height, device=device), torch.arange(width, device=device))
    canvas = torch.zeros((height, width, 3), device=device)

    for idx in sorted_idx:

```

```

        canvas = canvas.clone()
        mu = torch.stack([screen_x[idx], screen_y[idx]])
        sigma = cov2d[idx]
        color = colors[idx]
        opacity = opacities[idx]

        try: inv_sigma = torch.inverse(sigma)
        except RuntimeError: continue

        radius = 3.0 * torch.sqrt(torch.max(torch.abs(sigma)))
        radius = torch.clamp(radius, min=1.0, max=width)

        min_x = int(torch.clamp(mu[0] - radius, 0, width).item())
        max_x = int(torch.clamp(mu[0] + radius, 0, width).item())
        min_y = int(torch.clamp(mu[1] - radius, 0, height).item())
        max_y = int(torch.clamp(mu[1] + radius, 0, height).item())

        if max_x <= min_x or max_y <= min_y: continue

        grid_x_slice = x_grid[min_y:max_y, min_x:max_x].float()
        grid_y_slice = y_grid[min_y:max_y, min_x:max_x].float()
        dx = grid_x_slice - mu[0]; dy = grid_y_slice - mu[1]

        power = -0.5 * (inv_sigma[0, 0] * dx * dx + 2 * inv_sigma[0, 1] * dx *
        alpha_splat = torch.exp(power) * opacity
        alpha_splat = alpha_splat.unsqueeze(-1)

        current_bg = canvas[min_y:max_y, min_x:max_x].clone()
        canvas[min_y:max_y, min_x:max_x] = color * alpha_splat + current_bg

    return canvas

```

4. Loss Function (L1 + D-SSIM)

The paper uses a combination of L1 (Pixel-wise absolute difference) and D-SSIM (Structural Similarity).

- L1 ensures correct colors.
- SSIM ensures correct structure and texture, compensating for L1's tendency to be blurry.

```

In [ ]: def create_window(window_size, channel):
        # Gaussian window for SSIM
        def gaussian(window_size, sigma):
            gauss = torch.Tensor([math.exp(-(x - window_size//2)**2/float(2*sigma**2)) for x in range(window_size)])
            return gauss/gauss.sum()

        _1D_window = gaussian(window_size, 1.5).unsqueeze(1)
        _2D_window = _1D_window.mm(_1D_window.t()).float().unsqueeze(0).unsqueeze(0)
        window = _2D_window.expand(channel, 1, window_size, window_size).contiguous
        return window.to(device)

    def ssim(img1, img2, window_size=11, size_average=True):

```

```

# Standard SSIM implementation for PyTorch
channel = img1.size(-3)
window = create_window(window_size, channel)

mu1 = F.conv2d(img1, window, padding=window_size//2, groups=channel)
mu2 = F.conv2d(img2, window, padding=window_size//2, groups=channel)

mu1_sq = mu1.pow(2); mu2_sq = mu2.pow(2); mu1_mu2 = mu1*mu2

sigma1_sq = F.conv2d(img1*img1, window, padding=window_size//2, groups=channel)
sigma2_sq = F.conv2d(img2*img2, window, padding=window_size//2, groups=channel)
sigma12 = F.conv2d(img1*img2, window, padding=window_size//2, groups=channel)

C1 = 0.01**2; C2 = 0.03**2

ssim_map = ((2*mu1_mu2 + C1)*(2*sigma12 + C2))/((mu1_sq + mu2_sq + C1)*(sigma1_sq + sigma2_sq + C2))

if size_average: return ssim_map.mean()
else: return ssim_map.mean(1).mean(1).mean(1)

def calc_loss(render, target, lambda_dssim=0.2):
    # Loss = (1 - lambda) * L1 + lambda * D-SSIM
    # Rearrange images to (N, C, H, W) for SSIM
    render_d = render.permute(2, 0, 1).unsqueeze(0)
    target_d = target.permute(2, 0, 1).unsqueeze(0)

    l1 = torch.abs(render - target).mean()
    d_ssim = 1.0 - ssim(render_d, target_d)

    return (1.0 - lambda_dssim) * l1 + lambda_dssim * d_ssim

```

```

In [ ]: class SimpleGaussianModel:
    def __init__(self, N=10):
        self._xyz = nn.Parameter(torch.rand(N, 3, device=device) * 2.0 - 1.0)
        self._rotation = nn.Parameter(torch.rand(N, 4, device=device)); self
        self._scale = nn.Parameter(torch.ones(N, 3, device=device) * -2.0)
        self._opacity = nn.Parameter(torch.zeros(N, 1, device=device))
        self._color = nn.Parameter(torch.zeros(N, 3, device=device))

        self.xyz_gradient_accum = torch.zeros(N, device=device)
        self.denom = torch.zeros(N, device=device)

    def parameters(self):
        return [self._xyz, self._rotation, self._scale, self._opacity, self._color]

    def reset_stats(self):
        self.xyz_gradient_accum = torch.zeros(self._xyz.shape[0], device=device)
        self.denom = torch.zeros(self._xyz.shape[0], device=device)

    def densify_and_prune(self, max_grad=0.0002, min_opacity=0.05, extent=2.0):
        grads = self.xyz_gradient_accum / self.denom
        grads[grads.isnan()] = 0.0

        scales = torch.exp(self._scale)
        current_ops = torch.sigmoid(self._opacity)

```

```

# Thresholds
max_scale = torch.max(scales, dim=1).values
mask_clone = (grads >= max_grad) & (max_scale <= grow_percent * exte
mask_split = (grads >= max_grad) & (max_scale > grow_percent * exte

# New Params Lists
new_xyz = [self._xyz[~mask_split]]
new_rot = [self._rotation[~mask_split]]
new_scale = [self._scale[~mask_split]]
new_op = [self._opacity[~mask_split]]
new_col = [self._color[~mask_split]]

# Clone
if mask_clone.any():
    new_xyz.append(self._xyz[mask_clone])
    new_rot.append(self._rotation[mask_clone])
    new_scale.append(self._scale[mask_clone])
    new_op.append(self._opacity[mask_clone])
    new_col.append(self._color[mask_clone])

# Split
if mask_split.any():
    stds = scales[mask_split]
    means = self._xyz[mask_split]
    samples = 2

    # Subsample
    noise = (torch.rand(means.repeat(samples,1).shape, device=device
    split_xyz = means.repeat(samples,1) + noise
    split_scale = self._scale[mask_split].repeat(samples, 1) - math.

    new_xyz.append(split_xyz)
    new_rot.append(self._rotation[mask_split].repeat(samples,1))
    new_scale.append(split_scale)
    new_op.append(self._opacity[mask_split].repeat(samples,1))
    new_col.append(self._color[mask_split].repeat(samples,1))

# Concat
self._xyz = nn.Parameter(torch.cat(new_xyz, dim=0))
self._rotation = nn.Parameter(torch.cat(new_rot, dim=0))
self._scale = nn.Parameter(torch.cat(new_scale, dim=0))
self._opacity = nn.Parameter(torch.cat(new_op, dim=0))
self._color = nn.Parameter(torch.cat(new_col, dim=0))

# Prune
current_ops = torch.sigmoid(self._opacity)
current_scales = torch.exp(self._scale)
mask_keep = (current_ops.squeeze() >= min_opacity) & (torch.max(curr

if not mask_keep.all():
    if mask_keep.sum() == 0 and self._xyz.shape[0] > 0: mask_keep[0]
    self._xyz = nn.Parameter(self._xyz[mask_keep])
    self._rotation = nn.Parameter(self._rotation[mask_keep])
    self._scale = nn.Parameter(self._scale[mask_keep])
    self._opacity = nn.Parameter(self._opacity[mask_keep])
    self._color = nn.Parameter(self._color[mask_keep])

```

```
self.reset_stats()
```

```
In [ ]: class SceneGenerator:
    def __init__(self, W=100, H=100):
        self.W, self.H = W, H
        self.cameras = [] # List of (view_matrix, target_image)

    def generate_ground_truth_gaussians(self):
        # Create 3 spheres (Red, Green, Blue) at offsets
        # Red Sphere
        N_sphere = 100
        r_xyz = torch.randn(N_sphere, 3, device=device) * 0.2 + torch.tensor(
        r_col = torch.tensor([1.0, 0.0, 0.0], device=device).repeat(N_sphere

        # Green Sphere
        g_xyz = torch.randn(N_sphere, 3, device=device) * 0.2 + torch.tensor(
        g_col = torch.tensor([0.0, 1.0, 0.0], device=device).repeat(N_sphere

        # Blue Sphere (Back)
        b_xyz = torch.randn(N_sphere, 3, device=device) * 0.2 + torch.tensor(
        b_col = torch.tensor([0.0, 0.0, 1.0], device=device).repeat(N_sphere

        all_xyz = torch.cat([r_xyz, g_xyz, b_xyz], dim=0)
        all_col = torch.cat([r_col, g_col, b_col], dim=0)

        # High opacity, small scale gt
        all_op = torch.ones(all_xyz.shape[0], 1, device=device)
        all_scale = torch.ones(all_xyz.shape[0], 3, device=device) * 0.05
        all_rot = torch.zeros(all_xyz.shape[0], 4, device=device); all_rot[:

        return all_xyz, all_col, all_op, all_scale, all_rot

    def look_at(self, eye, center, up):
        # Create View Matrix (World -> Camera)
        z = (eye - center); z = z / torch.norm(z)
        x = torch.cross(up, z); x = x / torch.norm(x)
        y = torch.cross(z, x)

        R = torch.stack([x, y, z])
        t = -torch.matmul(R, eye)

        view_mat = torch.eye(4, device=device)
        view_mat[:3, :3] = R
        view_mat[:3, 3] = t
        return view_mat

    def generate_dataset(self, num_views=8):
        print(f"Generating {num_views} synthetic views...")
        gt_xyz, gt_col, gt_op, gt_scale, gt_rot = self.generate_ground_truth

        fov_y = 60.0
        angle_step = 360.0 / num_views
        radius = 4.0

        for i in range(num_views):
```

```

angle = math.radians(i * angle_step)
eye = torch.tensor([math.cos(angle)*radius, 0.0, math.sin(angle)*radius], device=device)
center = torch.tensor([0.0, 0.0, 0.0], device=device)
up = torch.tensor([0.0, 1.0, 0.0], device=device)

view_mat = self.look_at(eye, center, up)

# Render GT
with torch.no_grad():
    gt_img = differentiable_rasterizer(
        gt_xyz, gt_col, gt_op, gt_scale, gt_rot,
        self.W, self.H, view_mat, fov_y=fov_y
    )
    self.cameras.append({'view': view_mat, 'image': gt_img.detach(), 'gt': gt_img})

# Plot a few
plt.figure(figsize=(10,3))
plt.subplot(1,3,1); plt.imshow(self.cameras[0]['image'].cpu()); plt.title('GT View 1')
plt.subplot(1,3,2); plt.imshow(self.cameras[num_views//3]['image'].cpu()); plt.title('GT View 2')
plt.subplot(1,3,3); plt.imshow(self.cameras[num_views//2]['image'].cpu()); plt.title('GT View 3')
plt.show()

scene = SceneGenerator(W=100, H=100)
scene.generate_dataset(num_views=16)

```

5. Multi-View Training

We now iterate through the generated cameras. Ideally, the model should learn to position Gaussians in 3D to satisfy all views simultaneously.

```

In [ ]: model = SimpleGaussianModel(N=50) # Init sparse
optimizer = torch.optim.Adam(model.parameters(), lr=0.005) # Lower LR for st

iterations = 1001
densify_interval = 200
history_loss = []

print("Starting Training...")
for i in range(iterations):
    # 1. Pick Random View
    cam_idx = random.randint(0, len(scene.cameras)-1)
    cam = scene.cameras[cam_idx]

    optimizer.zero_grad()

    # 2. Forward
    render = differentiable_rasterizer(
        model._xyz,
        torch.sigmoid(model._color),
        torch.sigmoid(model._opacity),
        torch.exp(model._scale),
        model._rotation,
        scene.W, scene.H,
        cam['view'], cam['fov_y']
    )

```

```

)

# 3. Loss (L1 + SSIM)
loss = calc_loss(render, cam['image'])
loss.backward()

# 4. Stats
with torch.no_grad():
    model.xyz_gradient_accum += torch.norm(model._xyz.grad, dim=1)
    model.denom += 1

optimizer.step()
history_loss.append(loss.item())

# 5. Adapt
if i > 0 and i % densify_interval == 0:
    model.densify_and_prune(max_grad=0.0002, extent=4.0)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.005)

if i % 100 == 0:
    print(f"Iter {i}, Loss: {loss.item():.4f}, Points: {model._xyz.shape}")

# Result Validation (View 0)
cam0 = scene.cameras[0]
with torch.no_grad():
    res = differentiable_rasterizer(
        model._xyz, torch.sigmoid(model._color), torch.sigmoid(model._opacity),
        torch.exp(model._scale), model._rotation, scene.W, scene.H, cam0['view']
    )

plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.imshow(cam0['image'].cpu()); plt.title("GT View 0")
plt.subplot(1,3,2); plt.imshow(res.cpu()); plt.title(f"Result View 0 ({model._xyz.shape})")
plt.subplot(1,3,3); plt.plot(history_loss); plt.title("Loss")
plt.show()

```